

SPRING BOOT

```
package com.example.springweb.controller;

import java.util.List;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.example.springweb.model.Students;
import com.example.springweb.service.StudentsService;

@RestController
@RequestMapping("/jpa")
public class StudentsController {

    StudentsService service;

    public StudentsController(StudentsService service){
        this.service = service;
    }

    @GetMapping("/list")
    public List<Students> getStudent(){
        return service.getAllStudent();
    }

    @PostMapping("/add")
    public Students addStudent(@RequestBody Students student){
        System.out.println(student.getName());
        System.out.println(student.getAge());
        System.out.println(student.getCourse());
        return service.addStudentBy(student);
    }

}
```

```
package com.example.springweb.service;

import java.util.List;

import org.springframework.stereotype.Service;
```

```

import com.example.springweb.Repository.StudentsRepository;
import com.example.springweb.model.Students;

@Service
public class StudentsService {

    StudentsRepository repo;
    //constructor injection
    public StudentsService(StudentsRepository repo){
        this.repo = repo;
    }

    public List<Students> getAllStudent(){

        return repo.findAll();
    }

    public Students addStudentBy(Students student){
        repo.save(student);
        return student;
    }
}

```

```

package com.example.springweb.Repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.example.springweb.model.Students;

public interface StudentsRepository extends JpaRepository<Students,Integer>{

}

```

```

package com.example.springweb.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

```

```
// workbench + server
```

```
@Entity
@Table(name = "student")
public class Students {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;
    private String name;
    private int age;
    private String course;

    public Students() {
    }
    public Students(int id, String name, int age, String course) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.course = course;
    }
    public int getId() {
        return id;
    }
    public String getName() {
        return name;
    }
    public int getAge() {
        return age;
    }
    public String getCourse() {
        return course;
    }
    public void setId(int id) {
        this.id = id;
    }
    public void setName(String name) {
        this.name = name;
    }
    public void setAge(int age) {
        this.age = age;
    }
    public void setCourse(String course) {
        this.course = course;
    }
}
```

```
}
```

```
spring.application.name=springweb

spring.datasource.url=jdbc:mysql://localhost:3306/spring_boot_harlm
spring.datasource.username=root
spring.datasource.password=raj123

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true
```

pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <scope>runtime</scope>
</dependency>
```

Theory

1. Controller Layer

- **@RestController** → Handles HTTP requests and returns JSON responses.
- **@RequestMapping("/jpa")** → Base path for endpoints.
- Endpoints:
 - **@GetMapping("/list")** → fetch all students.
 - **@PostMapping("/add")** → add a new student.
- Uses **constructor injection** for StudentsService.

2. Service Layer

- **@Service** → Encapsulates business logic.
- Delegates persistence operations to the repository.
- Example: `getAllStudent()` calls `repo.findAll()`.

3. Repository Layer

- **JpaRepository** → Provides CRUD methods (`findAll`, `save`, `findById`, `deleteById`).
- Extending `JpaRepository<Students, Integer>` automatically gives you database operations.

4. Model Layer

- **@Entity** → Marks class as a JPA entity.
- **@Table(name="student")** → Maps to database table.
- **@Id + @GeneratedValue** → Primary key with auto-increment.
- Fields: `id`, `name`, `age`, `course`.

5. Database Configuration

- Defined in `application.properties`:
 - `spring.datasource.url` → MySQL connection.
 - `spring.jpa.hibernate.ddl-auto=update` → auto-create/update schema.

- `spring.jpa.show-sql=true` → logs SQL queries.



Coding Round Questions

- Controller → What is the role of `@RestController`?
- Service → Why do we use `@Service`?
- Repository → What is `JpaRepository` and what methods does it provide?
- Entity → What is the purpose of `@Entity` and `@Table`?
- Dependency Injection → Why is constructor injection preferred?
- Hibernate → How does Hibernate integrate with Spring Boot?



Machine Coding Round Challenge

Problem: Student CRUD API with JPA

Requirements:

1. Extend your current setup to support full CRUD:
 - `GET /jpa/list` → list all students.
 - `GET /jpa/{id}` → get student by ID.
 - `POST /jpa/add` → add new student.
 - `PUT /jpa/{id}` → update student.
 - `DELETE /jpa/{id}` → delete student.
2. Use `JpaRepository` methods (`findById`, `save`, `deleteById`).
3. Return appropriate HTTP responses.